

Connector

Demo Suite Documentation

Six Demonstrations of Enterprise AI Governance

Demo	Focus
1	Enterprise Governance
2	Governed Coding Workflow
3	7 Most Dangerous Attacks
4	Stable Thinking Engine
5	Selective Context + Identity-Aware
6	Deterministic Tool Execution

Narrative Arc
Safe (Demo 3) → **Stable** (Demo 4)
→ **Private** (Demo 5) → **Executable** (Demo 6)

Contents

Introduction

The Connector Demo Suite comprises six interconnected demonstrations that showcase the full capabilities of the Connector platform for enterprise AI governance. Each demo focuses on a specific aspect while building upon the narrative of safe, stable, private, and executable AI systems.

Demo Contract

Every demo follows strict integrity rules:

- No fake IDs
- No fake receipts
- No mocked journal chains
- No silent fallback on missing auth
- Stop immediately if health/receipts/runtime IDs are missing
- Every claim marked: `proven_live`, `partial_live`, `not_proven_live`, or `stub_mode`

Demo 1: Enterprise Governance

1.1 Overview

Thesis: Live runtime proof of Connector’s enterprise governance capabilities.

The canonical enterprise demo is a **dual-proof demonstration** that shows both buyer-facing and operator-facing evidence. It proves where Connector sits, what remains unchanged, what Connector governs, and which product claims are proven live.

1.2 11-Slide Structure

1. **Live Runtime Contract** – What the running node guarantees
2. **Buyer Boundary** – Where Connector sits in the stack
3. **Claim Posture** – Explicit proven/unproven claims
4. **Live Decision Path** – End-to-end governed decision flow
5. **Boundary Enforcement** – Policy-driven allow/deny
6. **Tool Governance** – Blocked/allowed tools enforcement
7. **Operator Explain + Trace** – Evidence retrieval commands
8. **Evidence + Proof** – HMAC-chained audit trail
9. **Cost + Compliance** – Token budgets, HIPAA tracking
10. **Failure + Recovery + HITL** – Escalation and review queues
11. **Pilot Close** – Summary and next steps

1.3 Key Claims Proven

- **Governance:** Policy engine with deny-overrides
- **Audit:** HMAC-chained receipts with CID references
- **Cost Control:** Token budgets with kill switches
- **Compliance:** HIPAA/GDPR framework support
- **Human-in-the-Loop:** Review queues for low-confidence decisions

1.4 Operator Commands

```
# Bootstrap live state
python demos/demo.py bootstrap

# Run interactive slide deck
python demos/demo.py

# Run without waiting
python demos/demo.py --no-wait

# Check health
connectorctl health
```

Demo 2: Governed Coding Workflow

2.1 Overview

Thesis: Claude Code + Connector gateway + DeepSeek in an agentic control plane narrative.

This demo presents a calm, bounded, operator-led workflow instead of a freeform live-agent show. It demonstrates the proto Agent OS / control plane architecture that enterprise teams require.

2.2 Workflow Moments

1. **Preflight + Claude Code Shell Posture** – Establish governed environment
2. **Safe Governed Success** – Normal operation through Connector gateway
3. **Boundary Deny + Safe Alternative** – Policy blocks with substitutes
4. **Conditional Review Path** – Human-in-the-loop for edge cases
5. **Operator Proof Retrieval** – Evidence commands and traces

2.3 Live vs Orchestrated

Component	Type	Description
Health, Gateway	LIVE	Connector API endpoints
CLS Compile	LIVE	Contract language compilation
Agent + Memory	LIVE	Runtime lifecycle
Governed Completions	LIVE	Firewall + guard pipeline
Decision Recording	LIVE	Audit receipts
Path Allow/Deny	ORCHESTRATED	Demo policy evaluation
Safe Substitute	ORCHESTRATED	Scripted security narrative

[★] WOW Moment

Controlled Beta Posture: The demo explicitly states its boundaries. Every claim is labeled as LIVE (Connector API) or ORCHESTRATED (runner logic).

2.4 Usage

```
# Quick start via Makefile
make demo2

# Manual execution
CONNECTOR_DEV_MODE=1 python demos/demo2/workflow_demo.py

# With options
python demos/demo2/workflow_demo.py --no-wait
python demos/demo2/workflow_demo.py --raw-json
```

Demo 3: 7 Most Dangerous Attacks

3.1 Overview

Thesis: “Why you cannot run agents without Connector.”

This attack-driven demo sends **real attack payloads** through a live Connector node and reports what the platform actually does—block, flag, allow, or error. Nothing is simulated.

3.2 The 7 Attacks

#	Attack	Connector Defense
1	Prompt Injection → Tool Hijack	AgentFirewall + SemanticInjectionDetector
2	Memory Poisoning	Firewall write scoring + HMAC chain + CID integrity
3	Tool Escalation	blocked_tools list + PolicyEngine + MAC guard
4	Cost Explosion	Token budget + BudgetGate + kill switch
5	Hallucinated Action	Receipt chain—no receipt = no truth
6	PII Exfiltration	ContentGuard PII detector + HIPAA report
7	Instruction Drift	BehaviorAnalyzer + InstructionPlane validation

3.3 5 Layers of Raw Evidence

1. **Exact payload sent** — The attack string that hits the gateway
2. **Raw HTTP response** — Status code, body JSON, headers
3. **Raw firewall inspect** — All 5 guard pipeline layers evaluated
4. **Raw journal entries** — HMAC-chained audit trail
5. **Raw receipts / cost / compliance** — API responses, not claims

[!] KILLER MOMENT

Evidence, Not Logs: Every attack shows 5 layers of raw evidence. If the firewall flags but doesn’t block, the demo says so plainly.

3.4 Usage

```
# Quick start
make demo3

# Run all attacks
CONNECTOR_DEV_MODE=1 python demos/demo3/attack_demo.py --no-wait

# With raw JSON output
python demos/demo3/attack_demo.py --raw-json
```

Demo 4: Stable Thinking Engine

4.1 Overview

Thesis: “Connector turns AI from short-term guessing into long-term, stable, constraint-following reasoning.”

Demo 3 proved security. Demo 4 proves **cognitive governance**: memory stability, instruction fidelity, reasoning consistency, and execution discipline.

4.2 The 5 WOW Moments

[★] WOW Moment

WOW Moment 1 – Memory Graph Stability: Same knowledge reused correctly after 30+ writes. Contradiction detected, not hidden.

[★] WOW Moment

WOW Moment 2 – Instruction Survives Noise: After 30+ inputs, rule still applied correctly. Vanilla LLM forgot it.

[★] WOW Moment

WOW Moment 3 – Reasoning Consistency: Step 1 and final answer align. Same query twice → same reasoning path.

[!] KILLER MOMENT

WOW Moment 4 – Refusal to Hallucinate: System says “Cannot conclude with current data.” Vanilla LLM guesses.

[★] WOW Moment

WOW Moment 5 – Deterministic Behavior: System enforces constraints at 3 independent layers. Not probabilistic—structural.

4.3 Comparison: Vanilla LLM vs Connector

Capability	Vanilla LLM	Connector
Long context	degrades	stable (30+ pkts)
Instructions	drift at ~15 msgs	anchored (CID)
Reasoning	inconsistent	aligned (2× same)
Hallucination	frequent	refused + cited
Memory	flat chat history	structured graph
Output	probabilistic	governed + proven
Contradictions	silent overwrite	detected + flagged
Evidence	none	HMAC-chained CIDs

4.4 Usage

```
# Quick start
make demo4

# Run stability demo
python demos/demo4/stable_thinking.py --no-wait
```

Demo 5: Selective Context + Identity-Aware Execution

5.1 Overview

Thesis: “The LLM doesn’t need to know everything. It just needs to know enough—and Connector ensures the rest stays protected.”

This demo proves **precision without exposure**: the LLM reasons with minimal safe context while the system executes correctly for the specific user/entity.

5.2 Patient Scenario

Patient	Internal State	LLM Sees
Maria Santos	Full PHI: SSN, email, cardiac history	“Patient: cardiac risk, urgent”
John Carter	Full PHI: SSN, email, mild headache	“Patient: routine, standard”

5.3 7 Critical Moments

1. **Side-by-Side** — Internal vs LLM view comparison
2. **RAW LLM Payload** — Undeniable proof with cryptographic verification
3. **Same Query, Different Users** — LLM sees similar, system executes different
4. **Controlled Failure** — LLM suggests PII, Connector blocks
5. **Identity-Aware Execution** — Killer moment: correct execution per patient
6. **Operator Policy Control** — Runtime context-level changes
7. **Cryptographic Execution Proof** — SHA-256 hashes, phi_overlap: 0

[!] KILLER MOMENT

Identity-Aware Execution: LLM never knew WHO—but the system knew WHAT and FOR WHOM. data_exposed: [] — VERIFIED: Nothing sensitive leaked.

5.4 RAW LLM Payload Proof

```
Payload Analysis:
[X] "Maria"      -> NOT FOUND
[X] "Santos"     -> NOT FOUND
[X] "123-45-6789" -> NOT FOUND
[X] "maria.santos@email.com" -> NOT FOUND
```

```
Payload hash: a3f7b2c8d9e1f5a2 (verified)
```

5.5 Usage

```
# Quick start
make demo5

# Run privacy demo
python demos/demo5/selective_context_demo.py --no-wait

# Verify data minimization
connectorctl prove agent <pid> --claim data_minimization --forensic
```

Demo 6: Deterministic, Constrained Tool Execution

6.1 Overview

Thesis: “Today’s AI suggests actions. Connector executes them—correctly, safely, and predictably.”

This demo proves that Connector turns AI from a **suggestion engine** into a **controlled execution system**.

6.2 12 Capabilities Across 4 Phases

Phase 1 — Intent → Valid Action Mapping

1. Structured Intent Parsing
2. Tool Schema Enforcement
3. Argument Validation

Phase 2 — Execution Boundaries

4. Allowed Tools Only
5. Path / Scope Constraints
6. Parameter Constraints

Phase 3 — Multi-Step Execution Discipline

7. Step Sequencing Control
8. Dependency Awareness
9. No Shortcut Execution

Phase 4 — Determinism & Stability

10. Same Input → Same Action Plan
11. Replayability
12. Receipt-Based Execution Proof

6.3 DevOps Scenario

Service	web-api-prod
Config	/etc/services/web-api/config.yaml
Actions	validate_config, update_config, restart_service, check_health

[!] KILLER MOMENT

Determinism Test (KILLER MOMENT): Run same command twice—identical execution plan. Hash-verified determinism.

6.4 Critical Moments

1. **Tool Call is Structured** — Parsed intent → mapped tool with strict schema
2. **Dangerous Command Refused** — “Delete all logs” → DENY with safe alternative
3. **Multi-step Workflow Enforced** — validate → update → restart sequence
4. **No Shortcut Allowed** — “Restart without update” → BLOCKED
5. **Determinism Test** — Same input → same execution plan
6. **Replay + Proof** — connectorctl prove -exec with step-by-step receipts

6.5 Comparison: Today's AI vs Connector

Capability	Today's AI	Connector
Tool calls	flexible but risky	strict + validated
Execution	probabilistic	deterministic
Workflow	optional	enforced
Safety	prompt-based	system-level
Replay	impossible	built-in
Audit	weak	strong

6.6 Usage

```
# Quick start
make demo6

# Run execution demo
python demos/demo6/deterministic_execution_demo.py --no-wait

# Prove execution
connectorctl prove --exec <execution_id>
```

Appendix: Script Reference

6.7 Individual Demo Scripts

Script	Purpose	Commands
run_demo1.sh	Enterprise demo	bootstrap, preflight, run, export
run_demo2.sh	Coding workflow	bootstrap, preflight, run, stop
run_demo3.sh	Security attacks	bootstrap, preflight, run, stop
run_demo4.sh	Stability	bootstrap, preflight, run, structure
run_demo5.sh	Privacy	bootstrap, preflight, run, scenario
run_demo6.sh	Execution	bootstrap, preflight, run, scenario

6.8 Master Script Commands

```
# List all demos
./run_all_demos.sh list

# Run specific demo
./run_all_demos.sh run 3

# Run all demos
./run_all_demos.sh run

# Export evidence
./run_all_demos.sh export

# Check status
./run_all_demos.sh status

# Stop all demo nodes
./run_all_demos.sh stop

# Show summary
./run_all_demos.sh summary
```

6.9 Environment Variables

Variable	Required	Default
CONNECTOR_URL	No	http://localhost:9091
CONNECTOR_API_KEY	No	dev-token (dev mode)
DEEPSEEK_API_KEY	For LLM demos	—
CONNECTOR_DEV_MODE	No	0 (disabled)
DEMO*_EXPORT_DIR	No	./evidence